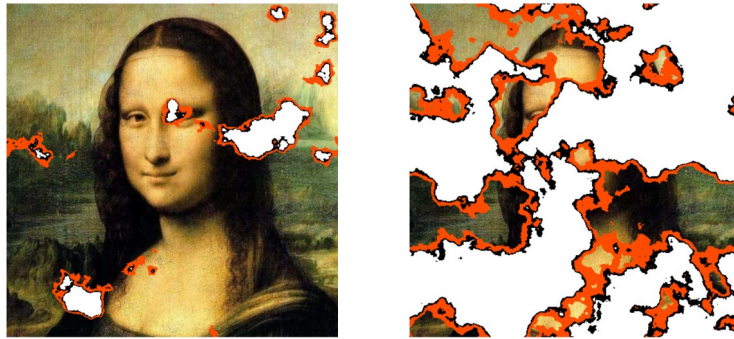

Burn (burn.*)

Escriu un **vertex shader** i un **fragment shader** que simulin un efecte de paper cremant. El VS sols ha de passar al FS les coordenades de textura, a banda d'escriure `gl_Position`. Al FS, texturarem l'objecte amb la imatge de la textura `pic`, i a mesura que passa el temps (representat com sempre pel valor del `uniform float time`), l'anirem destruint.

Concretament, en cada moment considerarem un llindar donat per `smoothstep(0,1,time/20+.2)`, i si el valor del canal vermell de la textura `foc1` és menor que aquest llindar, descartarem el fragment. Si aquest valor de la textura de `foc1` està entre el llindar i el llindar incrementat un 5%, el fragment serà de color negre, i si està entre el llindar augmentat en un 5% i el llindar augmentat un 15%, de color `rgb(1.0, 0.3, 0.0)`. Altrament, el fragment agafarà el color de la textura `pic`.

Aquí veieu la imatge de la Gioconda, en dos instants diferents (`time = 3`, i `time=5` fent servir les textures `gioconda.png` i `fbm.png`):



Identificadors obligatoris:

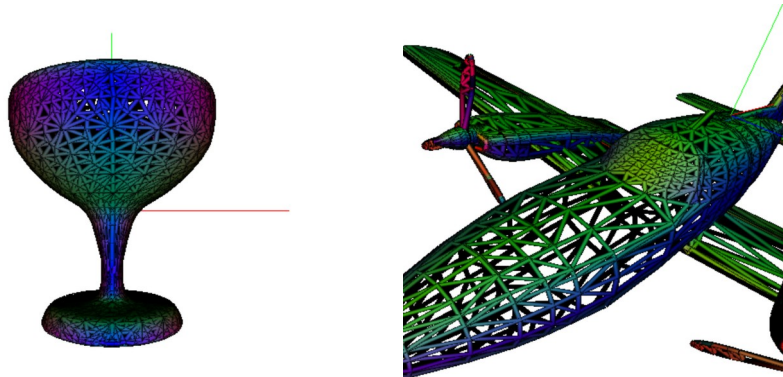
```
burn.vert, burn.frag
uniform float time;
uniform sampler2D pic;
uniform sampler2D foc1;
```

© Universitat Politècnica de Catalunya-BarcelonaTech, 2025. All rights reserved.

Thick Wires (thickWires.*)

Programa un *geometry shader* que visualitzi els models en filferros, però on cada «filferro» es veurà com un cilindre amb gruix. Per fer-ho, el GS rebrà del VS les coordenades dels vèrtexs en coordenades de clipping, i els colors associats a cada vèrtex (tal com els calcula el VS per defecte). Hauràs de passar les *coordenades de clipping* a *coordenades normalitzades de dispositiu* (NDC), i per cada triangle del model, el GS emetrà tres quads (dibuixats com a *triangle strips* de 4 vèrtexs), un per cada aresta. Donada l'aresta AB, els quatre vèrtexs els obtindrem desplaçant A i B en la direcció normal a l'aresta en la **projecció en pantalla** una distància igual a 0.01 en NDC, tant en la direcció positiva com en la negativa de la normal (recordem que en dimensió 2, un vector normal al vector (x, y) és el $(-y, x)$). És a dir que si la normal unitària que has calculat per AB és n , aleshores els vèrtexs són a $A \pm 0.01 * n$ i a $B \pm 0.01 * n$ (si tot està en NDC). A cada nou vèrtex li donaràs el color del seu “vèrtex original” (és a dir l'A o el B), i un atribut addicional de tipus **float** que valdrà 1 quan el vèrtex s'ha originat sumant la normal, i -1 quan s'ha obtingut restant-la.

D'aquesta forma, haurem dibuixat uns «billboards» associats a les arestes. Per completar la tasca, el FS haurà de colorar els fragments del color interpolat que rep del GS, modulats pel factor escalar que hem afegit com atribut als vèrtexs, i que representa la distància a l'aresta original. Si el valor interpolat d'aquest atribut és d , aleshores multiplicarem el color per $\cos\left(d\frac{\pi}{2}\right)$, on podeu prendre $\pi=3.141592$. Així, els billboards semblaran cilindres:



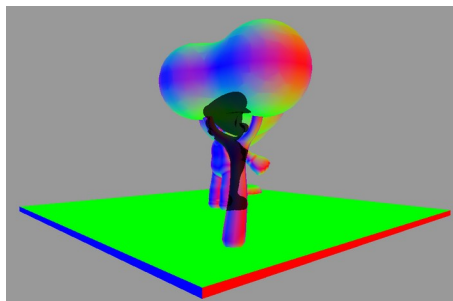
Identificadors obligatoris(tot minúscules):

`thickWires.vert`, `thickwires.geom`, `thickWires.frag`

Player XRay (playerXray.*)

Us proporcionem un render plugin que implementa parcialment una tècnica per tal de pintar l'últim objecte de l'escena de tal forma que sigui visible tot i que estigui oclòs per un altre objecte. És a dir, les parts no ocloses de l'objecte les pintarem utilitzant el shader per defecte. En canvi, les parts ocloses, les pintarem enfosquant el color d'aquella regió.

El codi proporcionat ja configura una escena amb múltiples objectes, on l'últim objecte és el model del Mario. L'efecte desitjat és el següent:



En la imatge podem veure el model complet del Mario tot i que està parcialment oclòs per l'arbre davant seu.

ÀVIS: Tal com us donem el codi, el model del Mario no es pintarà, és una de les tasques a fer.

Per tal d'aconseguir aquest efecte el plugin fa els següents passos:

Pas 1: Pintar tots els objectes de l'escena menys l'últim amb el shader per defecte.

Pas 2: Sense modificar el color buffer ni el depth buffer, pintar l'últim objecte per tal que el stencil buffer sigui 1 en els fragments on el model no ha passat el Z-test.

Pas 3: Pintar les parts occludes (on el stencil buffer té un valor de 1) amb el shader personalitzat sense utilitzar el Z-test i amb alpha blending activat, per tal que s'enfosqueixi el contingut del color buffer en aquella regió.

Pas 4: Pintar l'últim objecte amb el shader per defecte, per tal de pintar les parts no ocloses.

Les tasques a completar són les següents:

- Completar la funció `pintarUltimObjecte`, per tal que pinti l'últim objecte de l'escena.
- Configurar correctament el stencil buffer en el **Pas 2** i **Pas 3**.
- Carregar el vertex i fragment shader anomenats `uColor.vert` i `uColor.frag` en la funció `onPluguinLoad`.
- Fer servir el shader carregat en el **Pas 3**. Recordeu configurar correctament els uniforms necessaris.

Aquestes tasques també es troben especificades en el codi amb comentaris que comencen amb `// TODO:`, a **prop** d'on s'ha d'afegir el codi corresponent.

IMPORTANT: Heu d'entregar l'exercici com un zip o tar amb la carpeta **PlayerXRay** amb la vostra solució completa (hem de poder incloure aquesta carpeta a la carpeta de plugins del viewer i poder-lo executar amb el GLarenaPL). No incloure la carpeta *build* o el fitxer *Makefile* (podeu executar `make distclean` dins de la carpeta del plugin per a eliminar-les).